

# Software Engineering

## Lecture 1: Introduction to Software Engineering

**Instructor: Professor Nien-Lin Hsueh (with Gemini AI)**

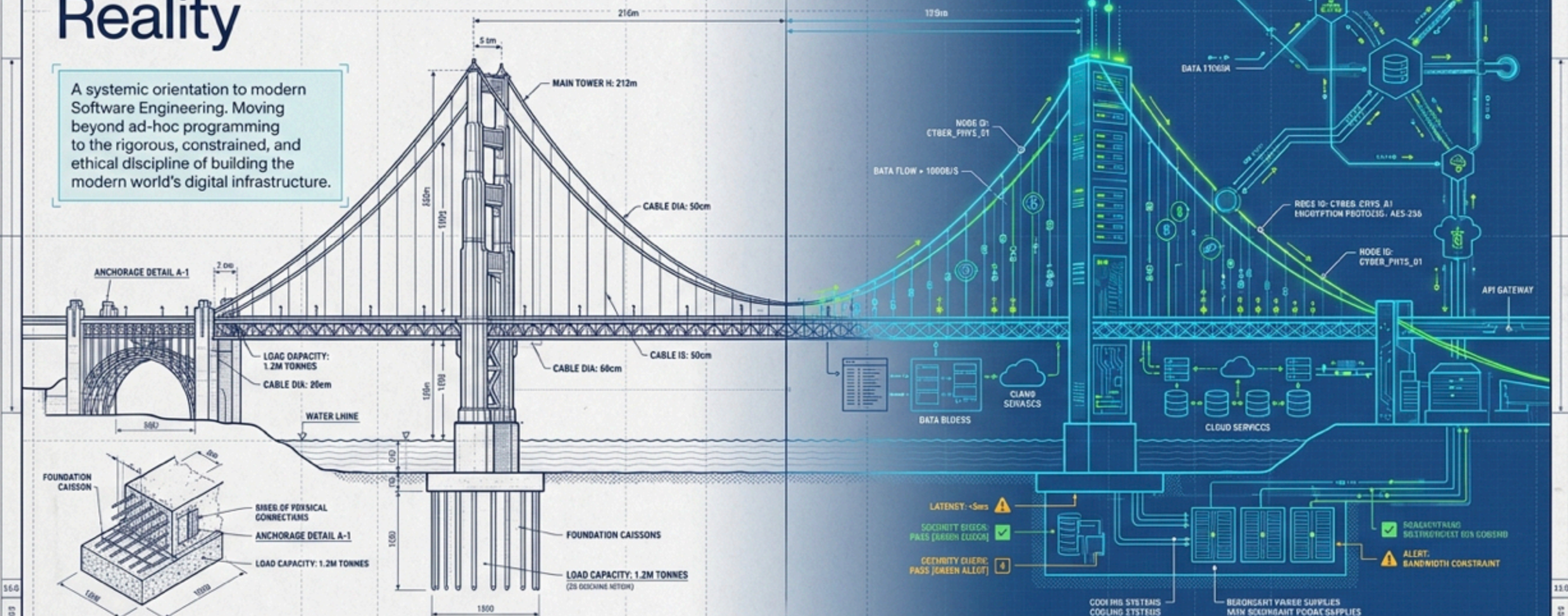
Department of Information Engineering and Computer Science  
Feng Chia University

# Chapter 1: Roadmap & Key Topics

- **1.1 The Technological Arc:** From Mechanization to Ubiquitous AI
- **1.2 The Genesis of SE & The "Software Crisis":** NATO 1968 & Catastrophic Failures
- **1.3 Demystifying Software:** Beyond Source Code (Programs, Data, SOP, Docs)
- **1.4 ISO 9126 Quality Model:** 6 Characteristics & Operational Sub-Attributes
- **1.5 Modern Software Landscape:** Web, Mobile, ERP, Embedded, AI/ML
- **1.6 What is SE?:** Definition, Process, Constraints, 4 Core Activities, Principles & Myths
- **1.7 Modern Toolchains & AI:** CI/CD Pipelines & AI Benefits/Risks Matrix
- **1.8 Professional Ethics & Dark Patterns:** ACM/IEEE Code & Deceptive UI
- **1.9 FAQ & Conceptual Recap**

# From Code to Cyber-Physical Reality Reality

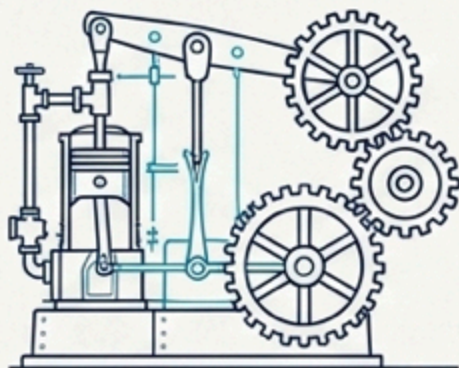
A systemic orientation to modern Software Engineering. Moving beyond ad-hoc programming to the rigorous, constrained, and ethical discipline of building the modern world's digital infrastructure.



# The Evolution of Industrial Complexity

## Industry 1.0 (Mechanization)

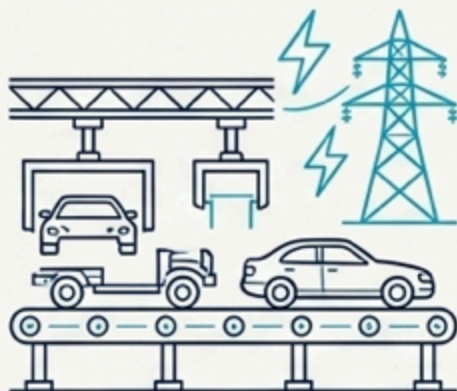
Late 18th-Mid 19th Century



Steam and water power.

## Industry 2.0 (Mass Production)

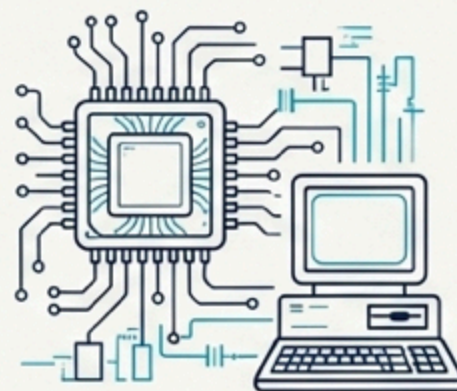
Late 19th-Early 20th Century



Electricity and assembly lines.

## Industry 3.0 (Digital Revolution)

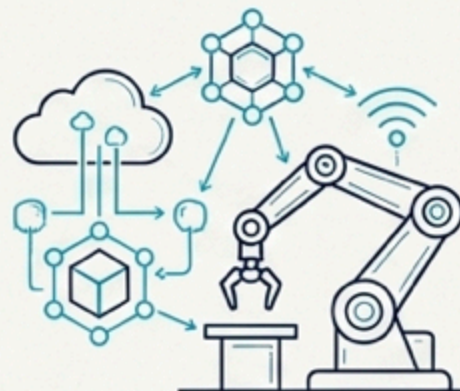
Mid-Late 20th Century



Computers, semiconductors, and PLCs.

## Industry 4.0 (Cyber-Physical)

Present



IoT, AI, Cloud Computing, Advanced Robotics

### Role of Software

1.0: Non-existent

2.0: Absent  
(electro-mechanical)

3.0: Emergent  
(Dawn of Software Engineering)

4.0: Pervasive  
(Intelligent backbone)

# 1.1 The Four Industrial Eras

- **Industry 1.0 — Mechanization (Late 18th - Mid 19th C.):**
  - Steam and water power replace physical muscle; rise of centralized factories.
  - *Software's Role:* Non-existent. Purely physical mechanics.
- **Industry 2.0 — Mass Production (Late 19th - Early 20th C.):**
  - Electricity, moving assembly lines, standardization of manufactured goods.
  - *Software's Role:* Non-existent. Automation was hardwired electro-mechanical.
- **Industry 3.0 — Digital Automation (Mid to Late 20th C.):**
  - Semiconductors, microprocessors, PLCs, and PCs.
  - *Software's Role:* **Emerged as a distinct discipline** to control hardware and process data.
- **Industry 4.0 — Connectivity, Cloud, & AI (21st C. - Present):**
  - Cyber-physical systems, IoT, Cloud, LLMs, and autonomous systems.
  - *Software's Role:* **The central infrastructure** powering modern civilization.

# 1.1 Interactive Activity: Classroom Poll & Discussion

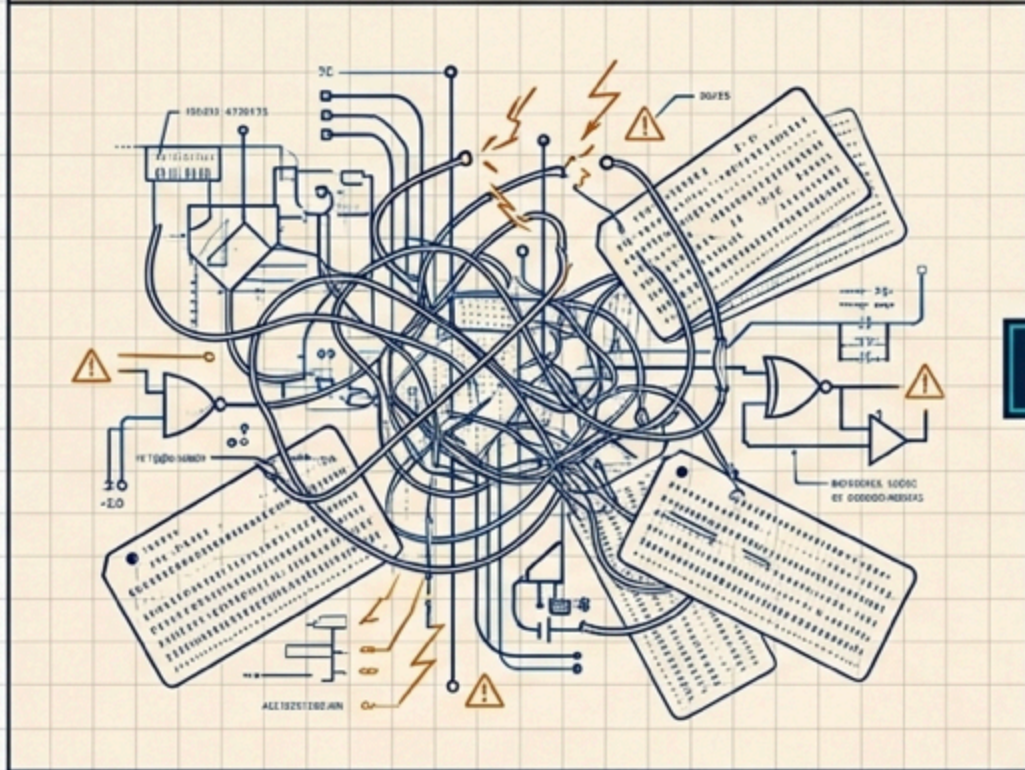
## Classroom Poll & Discussion:

- **Poll:** What percentage of systems you interact with daily run on deterministic Industry 3.0 logic vs. adaptive, connected Industry 4.0 AI?
- **Pair Discussion:** Identify one legacy manual system (e.g., campus parking, hospital triage, public transit dispatch). What unique software engineering challenges arise when transitioning it to Industry 4.0?



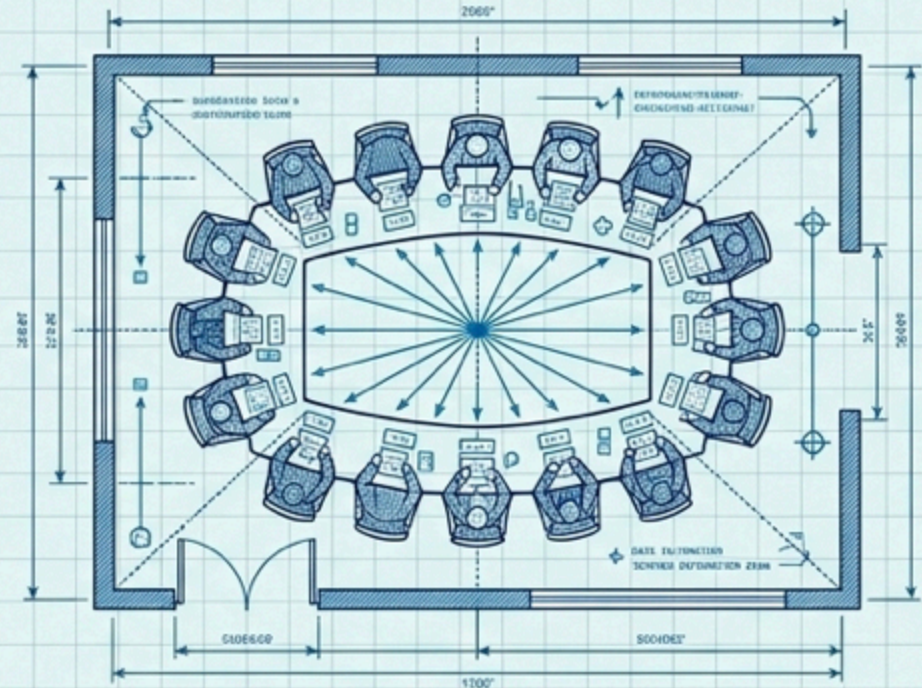
# Surviving the Software Crisis

## The Crisis (1960s/70s)



Rapidly scaling complexity exposed the flaws of custom, ad-hoc programming. Projects consistently suffered from massive budget overruns, late deliveries, and outright system failures.

## The Genesis (1968 NATO Conferences)



Experts gathered to establish a formal discipline—borrowing the predictability, safety margins, and systematic planning of physical engineering to manage digital complexity.

## 1.2 The Software Crisis of the Late 1960s

- As hardware costs plummeted, software demand and complexity exploded.
- **The Symptoms of Crisis:**
  - Chronic budget overruns (often 3x–4x initial estimates).
  - Critical project delays and abandoned deliveries.
  - Severe defects, system crashes, and unmaintainability.
- **The Core Problem:** Informal, craft-like programming does not scale to large teams and complex systems.

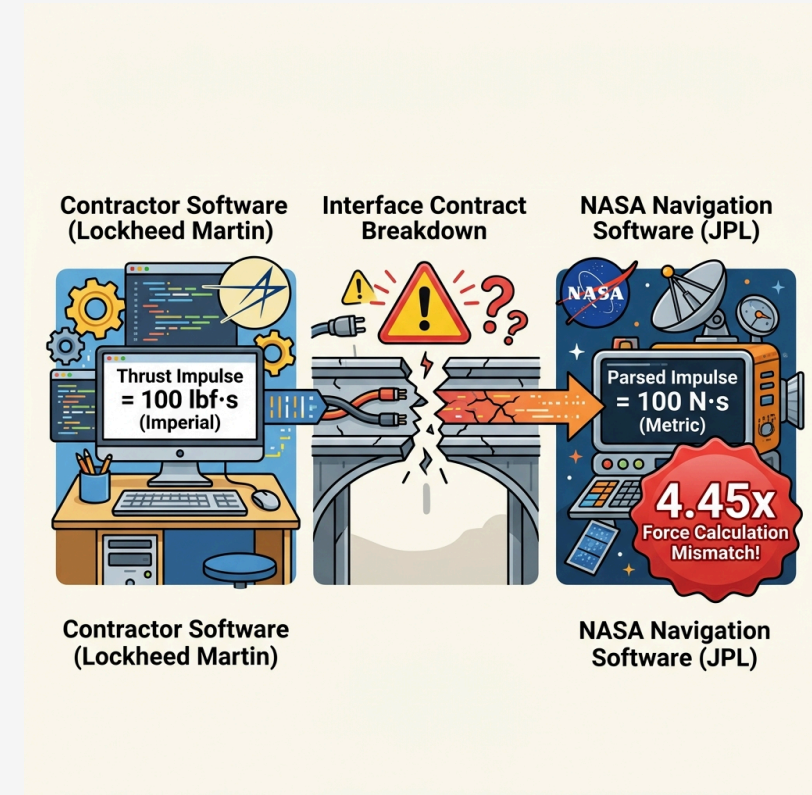


## 1.2 The 1968 NATO Garmisch Conference

- **October 1968 in Garmisch, Germany:**
  - 50 leading computer scientists and industry managers convened.
  - Formally established the term "**Software Engineering**".
- **The Mission:**
  - Transition software development from an undisciplined, artisanal craft into a **formal engineering discipline**.
  - Establish structured methodologies, rigorous cost estimation, formal verification, and project management.

# 1.2 The High Cost of Software Failure

- **Nagoya Airbus A300 Crash (1994):**
  - Autopilot Go-Around mode stayed active; fought pilot's manual steering, leading to trim nose-up stall and 264 fatalities.
- **Mars Climate Orbiter (1999):**
  - \$327M probe lost because ground software calculated thrust in imperial  $lbf \cdot s$ , while onboard computer expected metric  $N \cdot s$ .
- **Ariane 5 Flight 501 (1996):**
  - 64-bit float representation of horizontal velocity overflowed 16-bit signed integer ( $> 32,767$ ), crashing processors in 37 seconds.



# Concept Check: The Software Crisis (CCQ 1)

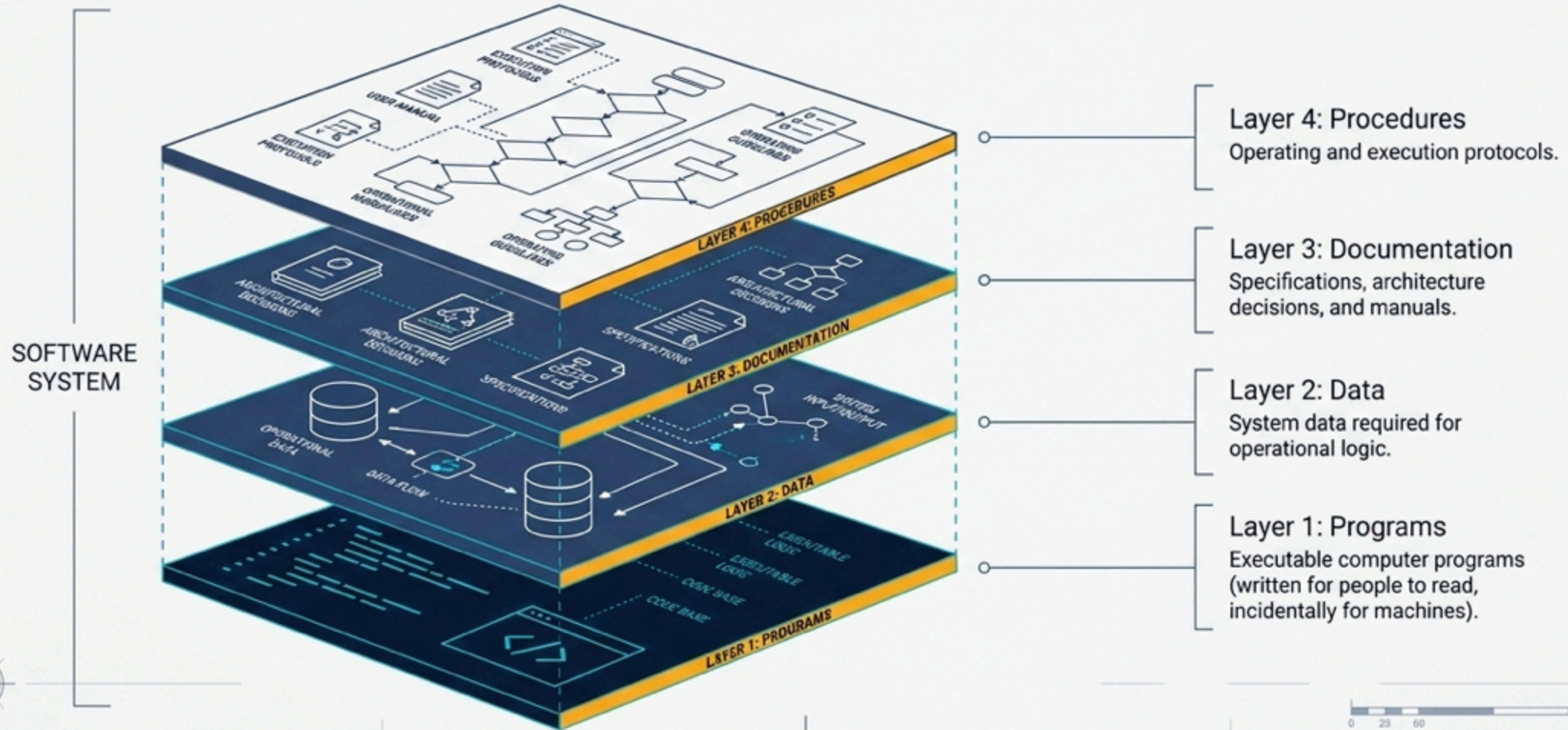
**Why couldn't the 1968 Software Crisis be resolved simply by purchasing faster computer hardware or larger memory?**

- **A.** Hardware manufacturing stopped advancing in the late 1960s.
- **B.** The crisis was fundamentally a cognitive and organizational problem of intellectual complexity, system architecture, and communication overhead, which faster hardware only amplified.
- **C.** Programming languages lacked mathematical calculation capabilities.
- **D.** Hardware was incompatible with cloud infrastructure.



# The Anatomy of Software Exceeds Source Code

A failure in operational procedures is a failure of the software system. Physical hardware is explicitly excluded; software encompasses the intangible logic governing the system.



## 1.3 The IEEE Anatomy of Software

**Software (IEEE Standard):** *Computer programs, procedures, and possibly associated documentation and data pertaining to the operation of a computer system.* ”

- **1. Programs:** Executable binaries and source code files (Python, Java, C++, TypeScript).
- **2. Data & Schemas:** Database tables, config files, AI weights (e.g. Knight Capital's config error).
- **3. Operational Procedures:** Deployment scripts, backups, recovery runbooks (e.g. GitLab's backup failure).
- **4. Documentation:** Architecture ADDs, API specs (OpenAPI), user guides (e.g. Therac-25 undocumented bugs).

## 1.3 The Software Iceberg Trap

- Many software projects fail because developers and managers fall into the trap of only focusing on the visible "tip" of the software iceberg—the **Source Code**.
- They view progress solely by lines of code written, ignoring database schemas, operational backup runbooks, and design API documentation.
- Without all four pillars, a system is not "professional software"—it is merely a brittle program that cannot be deployed or maintained safely.

## Concept Check: The Software Definition (CCQ 2)

**According to the IEEE standard definition of software, which of the following is NOT considered a component of software?**

- **A.** Executable computer programs and source code files.
- **B.** System database schemas and configuration files.
- **C.** CPU processor hardware and physical memory units.
- **D.** Software installation and deployment procedures.



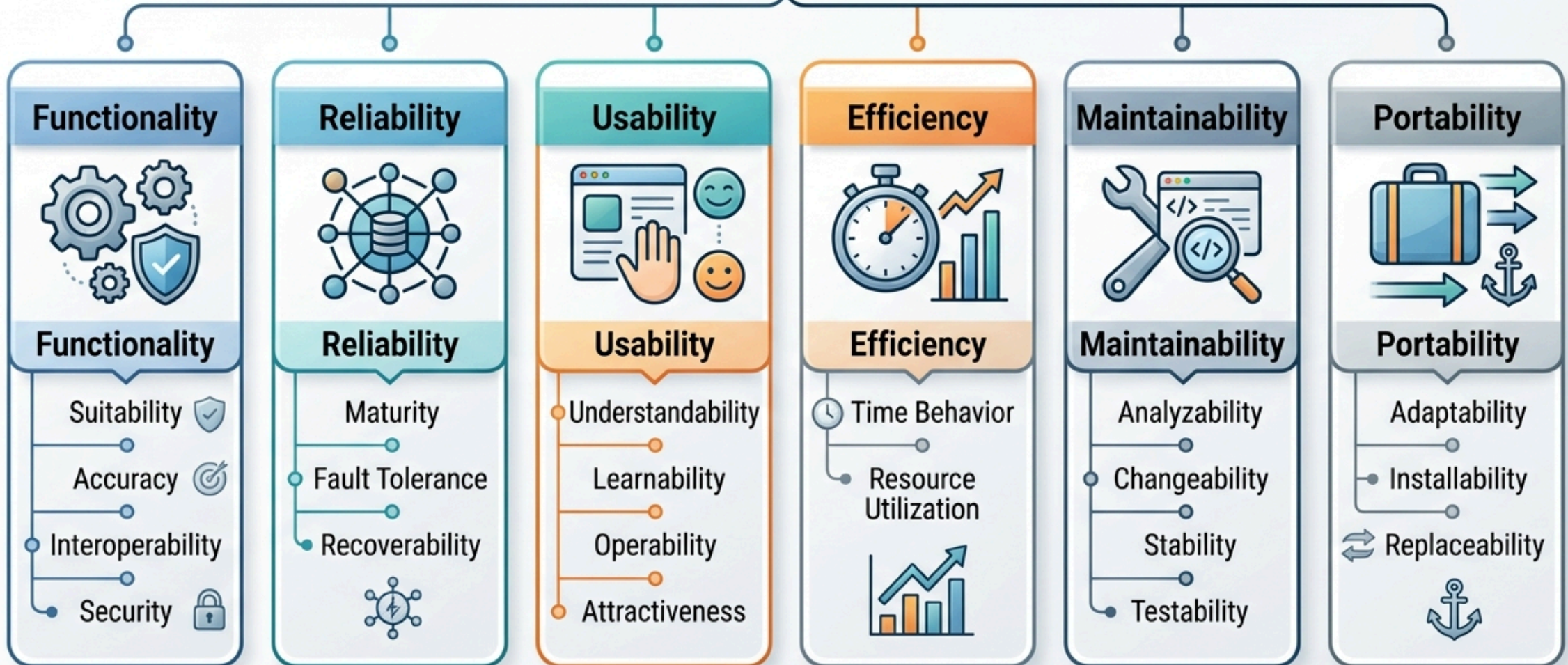
## 1.3 Interactive Activity: Pair Discussion

### Pair Discussion: The Software Iceberg in Action

- Select a popular service: **Google Maps**, **Uber**, or **Spotify**.
- Identify at least one specific artifact for each of the 4 pillars:  
1. *Program* | 2. *Data* | 3. *Procedure* | 4. *Documentation*
- **Debate:** If an engineering team lost all database schemas and operational runbooks, could they restore production using only source code?



# ISO/IEC 9126 Software Quality Model



## 1.4 ISO 9126: 6 Characteristics & Sub-Attributes

- **Functionality:** Suitability, Accuracy, Interoperability, Security, Compliance.
- **Reliability:** Maturity, Fault Tolerance, Recoverability, Compliance.
- **Usability:** Understandability, Learnability, Operability, Attractiveness.
- **Efficiency:** Time Behavior (Latency / Throughput), Resource Utilization.
- **Maintainability:** Analyzability, Changeability, Stability, Testability.
- **Portability:** Adaptability, Installability, Co-existence, Replaceability.

## 1.4 ISO 9126 Sub-Attributes & Practical Examples

- **Fault Tolerance (Reliability):** Primary payment gateway times out → system automatically retries with backup gateway without crashing.
- **Recoverability (Reliability):** Database server crashes → Write-Ahead Log restores transactional consistency in  $< 30$  seconds.
- **Time Behavior (Efficiency):** E-commerce search API returns 99% of query responses in  $< 80$  ms (p99 latency).
- **Testability (Maintainability):** Code structured with Dependency Injection so external APIs can be mocked easily in unit tests.
- **Installability (Portability):** Complete local microservices stack spins up in 60s via `docker compose up`.

## Concept Check: Software Quality Factors (CCQ 3)

**Which of the following matches a real-world software issue with its corresponding ISO 9126 quality characteristic?**

- **A.** A database query taking 15 seconds to return results → Maintainability (Testability)
- **B.** A system crash occurring when a third-party API goes offline → Reliability (Fault Tolerance)
- **C.** Developers struggling to write unit tests due to tight coupling → Portability (Adaptability)
- **D.** The application failing to run on a new macOS version → Usability (Operability)



## 1.4 Interactive Activity: Quality Trade-Off Poll

### Quality Trade-off Poll & Discussion:

- **System A:** Hospital ICU Automated Insulin Pump Controller
- **System B:** Mobile Casual Viral Game
- **Poll:** What are the top 2 non-negotiable ISO 9126 attributes for System A vs. System B?
- **Key Question:** Why is prioritizing *Time to Market* over *Fault Tolerance* fatal for System A, but acceptable for System B? In your opinion, what constraints or factors impair software quality or make high quality hard to achieve?



# YOUBIKE SMART BICYCLE SHARING SYSTEM DIAGRAM

## YOUBIKE SMART BICYCLE



## MOBILE APP

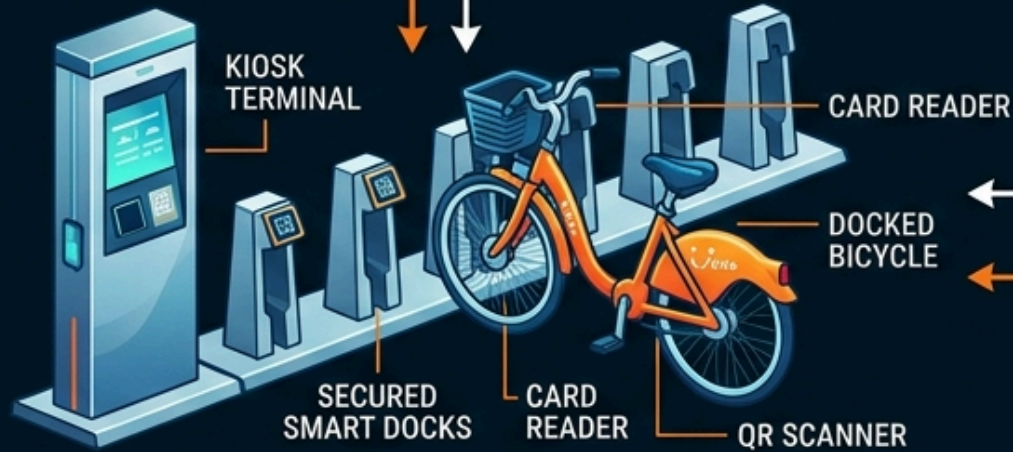


**CELLULAR/IoT**

**REAL-TIME LOCATION (GPS)  
BICYCLE HEALTH  
USAGE LOGS**

**BICYCLE  
LOCK/UNLOCK**

**USER  
RENTAL/RETURN**



**API/MQTT**

**STATION STATUS  
RENTAL DATA  
HEARTBEAT**

**USER AUTH  
BOOKING  
PAYMENT  
UNLOCK COMMAND**



**DOCKING STATION (DOCK)**

**BACKEND CLOUD SERVER**

## 1.5 Heterogeneous Systems: The YouBike Example

- A modern software system is rarely a single homogeneous program.
- **YouBike** combines multiple application domains in a single system:
  - **Embedded Firmware:** Bike IoT hub locks, docking stations (樁).
  - **Mobile App:** Touch-screen UI, booking, unlocking, map search.
  - **Web Application:** Official site, customer registration.
  - **Enterprise Backend:** Cloud database schemas, ACID transactions, billing.
- Successful software engineering requires coordinating these diverse domains.

## 1.5 Key Application Domains

- **Web & SaaS Platforms:** Elastic cloud, high availability, microservices (Slack, Netflix).
- **Mobile Applications:** Constrained battery, touch UI, intermittent networks (iOS/Android).
- **Enterprise ERP / CRM:** ACID transactions, complex business logic, data governance (SAP, Salesforce).
- **Embedded & IoT Firmware:** Strict real-time constraints, limited memory, zero-fail avionics & automotive.
- **AI & Machine Learning:** Data pipelines, vector databases, GPU acceleration, LLM inference.
- **Scientific & CAD:** Floating-point precision, simulation physics, hardware acceleration.

## 1.5 Interactive Activity: System Classification

### System Classification & Hybrid Architectures:

- Consider a modern **Connected Electric Vehicle (e.g., Tesla)**.
- Which application domains does it encompass?
  - Real-time embedded firmware for braking / motor control.
  - Touchscreen UI for navigation & entertainment.
  - Cloud-native backend for fleet telemetry and OTA updates.
  - Edge AI models for autonomous vision.
- **Discussion:** Why do update cadences differ dramatically across these subsystems?



# CODING



# SOFTWARE ENGINEERING

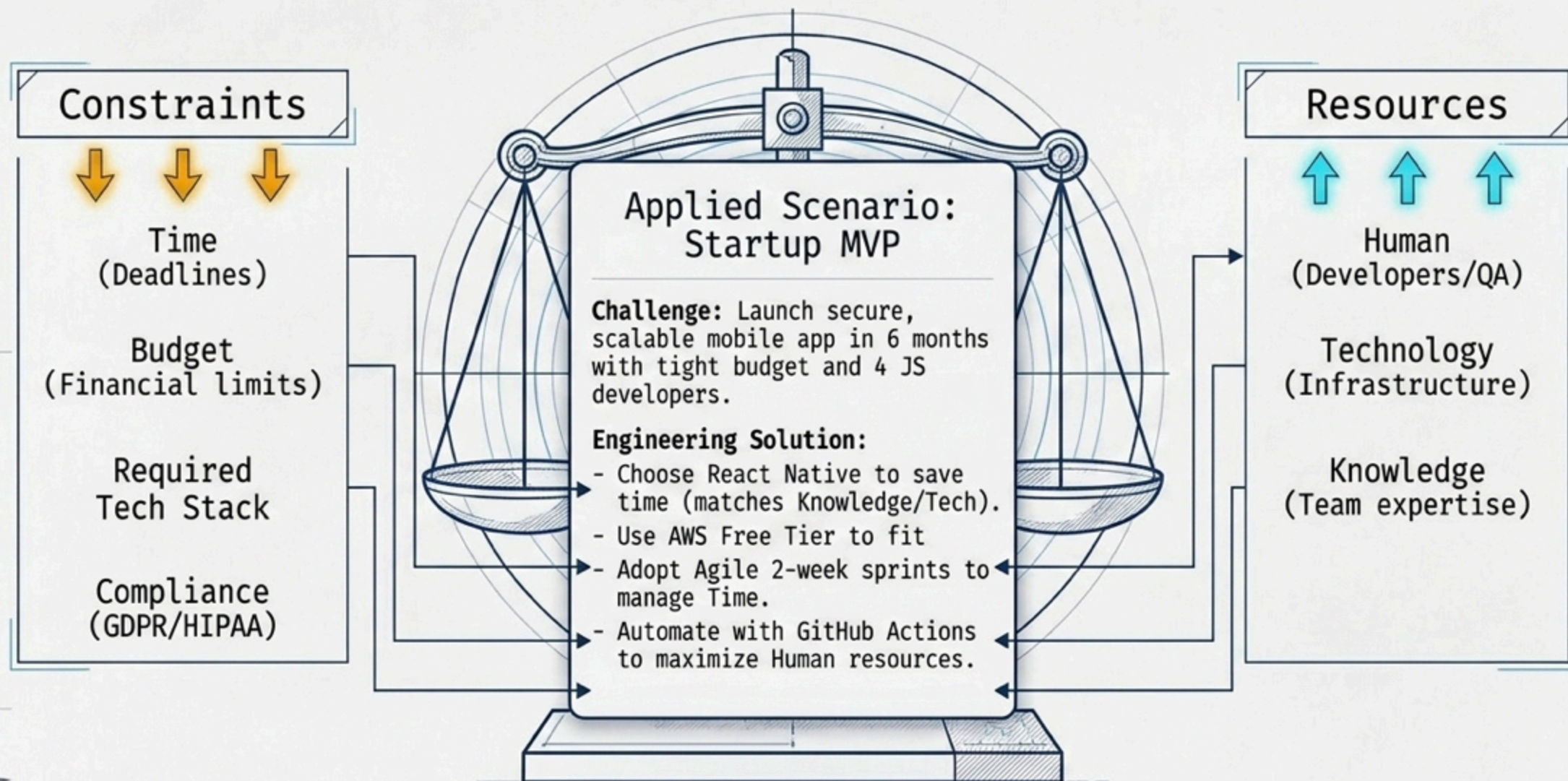


## 1.6 What is Software Engineering?

**Software Engineering:** *An engineering discipline concerned with all aspects of software production—from initial specification through to system maintenance and evolution.* „

- **1. Engineering Discipline (Work Under Constraints):** Engineering applies scientific rigor and heuristics to solve real human problems under **strict constraints using finite resources**.
- **2. All Aspects of Production (The Software Process):** Governed by a systematic **Software Engineering Process (SE Process)** that guides activities, roles, and deliverables reliably from inception to evolution.

# The Engineering Balancing Act



# 1.6 The Engineering Equation: Constraints vs. Resources

- **Constraints (The Limitations):**

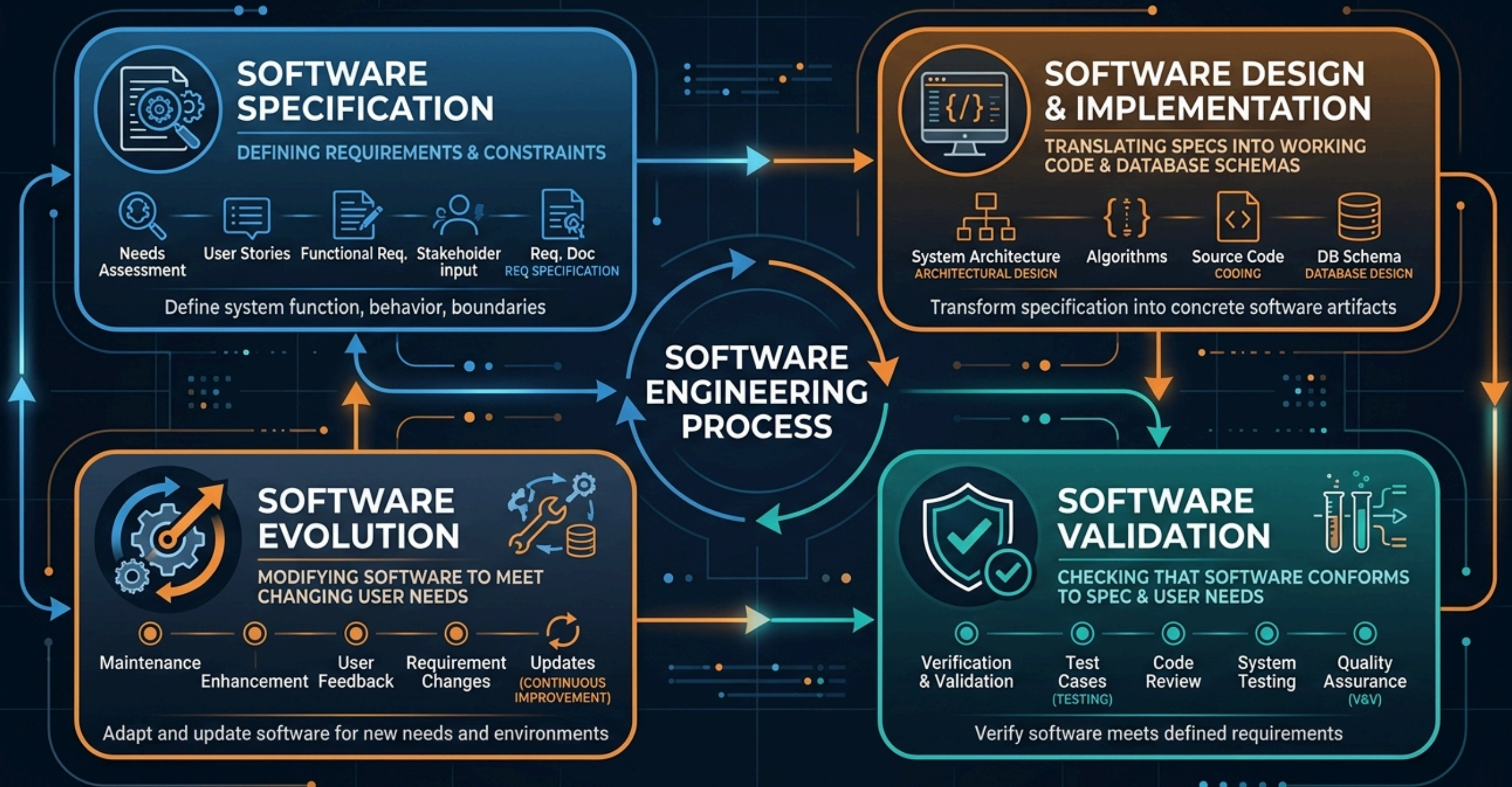
- **Time:** Hard deadlines, release windows.
- **Budget:** Developer salaries, cloud hosting bills, license fees.
- **Technology & Platform:** Legacy databases, mobile OS limits.
- **Regulations:** GDPR data privacy, HIPAA, PCI-DSS.

- **Resources (The Assets):**

- **Human:** Developer skill, QA engineers, UX designers.
- **Tools & Infrastructure:** Cloud compute, CI/CD pipelines, open-source libraries.
- **Domain Knowledge:** Understanding user workflows and business logic.

## 1.6 Case Study: Healthcare Startup MVP

- **Problem:** Launch a HIPAA-compliant telemedicine app in 6 months on a tight \$80k budget.
- **The Engineering Balancing Solution:**
  1. **Scope Prioritization:** Focus MVP strictly on video consults and booking; postpone insurance billing.
  2. **Cross-Platform Tooling:** Use Flutter/React Native for a single codebase (saves 40% effort).
  3. **Managed Backend:** Use HIPAA-compliant cloud BaaS instead of bare-metal servers.
  4. **Automated CI/CD:** unit tests on PRs, keeping QA overhead low.
- **Why Not "Perfect" Technical Solutions?**
  - Custom native apps and custom microservices are technically superior, but rejected because they violate constraints (\$80k budget, 6 months).
  - **Satisficing:** Find the solution that meets all constraints through trade-offs.



# 1.6 The 4 Universal Lifecycle Activities

| Activity                                  | Key Artifacts                                    | Risk If Neglected                                         |
|-------------------------------------------|--------------------------------------------------|-----------------------------------------------------------|
| <b>1. Specification</b><br>(Requirements) | User Stories, SRS, Acceptance Criteria           | Building the wrong product; 100x rework cost              |
| <b>2. Design &amp; Implementation</b>     | Architecture ADD, ERD, API Specs, Codebase       | Spaghetti code, unscalable architecture, technical debt   |
| <b>3. Validation</b> (Testing / V&V)      | Unit/Integration Tests, CI Reports, Bug Trackers | Critical production outages, data loss, security breaches |
| <b>4. Evolution</b> (Maintenance)         | Release Notes, DB Migrations, Post-Mortems       | Software rot, security vulnerabilities, obsolescence      |

## Concept Check: Core Activities (CCQ 4)

**Which of the following pairs correctly matches a specific software engineering action with its corresponding universal core activity?**

- **A.** Conducting stakeholder interviews to draft user stories → Software Specification
- **B.** Writing automated unit tests to mock database responses → Software Design & Implementation
- **C.** Refactoring database schemas to improve query speed → Software Validation
- **D.** Swapping a third-party payment API for a new gateway → Software Specification



## 1.6 Interactive Activity: Scenario Analysis

### Interactive Scenario: Which Activity Failed?

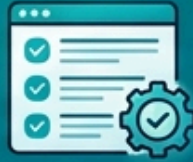
- **Scenario:** A startup built a lightning-fast in-app crypto wallet for a community grocery delivery app. The code had 100% test coverage and zero crashes. After launch, zero users adopted it because community shoppers exclusively preferred cash-on-delivery.
- **Question:** Which fundamental activity failed: *Specification*, *Design*, *Validation*, or *Evolution*?
- **Key Lesson:** Why can 100% code test coverage never compensate for a failure in Requirements Specification?



## • CORE ELEMENTS OF SOFTWARE ENGINEERING •



### Disciplines (Best Practices)



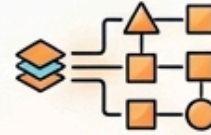
- Spec before design
- Design before code
- Interface-based
- Change management
- ADR (Architecture Decision Records)
- Code Reviews



### Principles (Core Foundations)



- Abstraction
- Modularity
- Anticipation of Change
- Captration
- Open-Closed (OCP)
- KISS (Keep It Simple, Stupid)
- POLA (Principle of Least Astonishment)



### Methods & Methodologies



- Waterfall
- Agile / Scrum
- Spiral Model
- TDD (Test-Driven Development)
- CI/CD & DevOps



### Heuristics & Guidelines



- Nielsen's 10 UX Heuristics
- SOLID design
- Clean Code
- DRY (Don't Repeat Yourself)
- YAGNI (You Ain't Gonna Need It)

# 1.6 Elements Encompassed by Software Engineering

- **Disciplines (Best Practices):** Spec before design, Design before code, Interface-based, Change management, ADRs, Code reviews, ...
- **Principles (Foundations):** Abstraction, Modularity, Anticipation of Change, Open-Closed (OCP), KISS, POLA, ...
- **Methods & Methodologies:** Waterfall, Agile / Scrum, Spiral, TDD, CI/CD & DevOps, ...
- **Heuristics & Guidelines:** Nielsen's 10 UX Heuristics, SOLID design, Clean Code, DRY, YAGNI, ...

# 1.6 Dispelling Common Software Myths

- **Myth 1: "We're behind schedule—let's add 5 developers to catch up."**
  - **Reality (Brooks's Law):** Adding manpower to a late project makes it later ( $O(n^2)$  communication overhead).
- **Myth 2: "Software is digital, so changing requirements late is cheap."**
  - **Reality:** Late changes invalidate database schemas and architectures, costing up to 100x more.
- **Myth 3: "Outsource the coding and we don't need technical management."**
  - **Reality:** Outsourcing requires rigorous technical governance.

# Myth: Changing requirements late is cheap!

Requirement change early during blueprint design (Cost: 1x)



Requirement change at the very end of construction (Cost: 100x)



## Concept Check: Brooks's Law (CCQ 5)

**A project is 3 weeks behind schedule with 2 weeks remaining before release. The manager hires 4 junior programmers to speed up progress. What will happen according to Brooks's Law?**

- **A.** The project will finish 1 week early.
- **B.** The project will be delayed further because senior engineers must spend time onboarding and mentoring new hires.
- **C.** The existing developers will code twice as fast.
- **D.** Communication complexity remains unchanged.



## 1.7 The Modern Engineering Toolchain

- **Version Control (Git):** Branching strategies, PR collaborative code review.
- **Modern IDEs (VS Code, IntelliJ):** Real-time linting, static analysis, refactoring.
- **CI/CD Pipelines (GitHub Actions, GitLab CI):** Automated compiling, unit testing, security scanning, and deployment upon every commit.
- **Testing & QA Suites:** Unit (PyTest), Integration, E2E (Playwright).
- **Observability & APM:** Telemetry, logs, OpenTelemetry, Sentry APM.
- **Containerization:** Docker containers, Kubernetes cluster management.
- **Project Management:** Sprint backlogs, bug tracking (Jira, Linear).
- **Static Code Analysis:** linting, code smells, vulnerabilities (SonarQube).
- **API Collaboration:** Design, mock, document, and test APIs (Postman).
- **Infrastructure as Code (IaC):** Cloud provisioning configs (Terraform).

## 1.7 AI in SE: Benefits vs. Risks Matrix

| Phase               | Benefits of AI                          | Risks & Drawbacks                               |
|---------------------|-----------------------------------------|-------------------------------------------------|
| <b>Requirements</b> | Drafts user stories, finds ambiguity    | Hallucinates constraints, misses tacit context  |
| <b>Architecture</b> | Recommends patterns, drafts ERDs        | Over-engineering, ignores latency/security SLAs |
| <b>Coding</b>       | Fast boilerplate, algorithm suggestions | "Vibe coding" bugs, security vulnerabilities    |
| <b>Testing</b>      | Generates synthetic edge-case tests     | Echo-chamber tests (validating buggy code)      |
| <b>Maintenance</b>  | Explains legacy code, drafts docs       | Silent regression bugs during refactoring       |

JUST COMPILE IT.  
TRUST THE VIBES!

CRITICAL  
SYSTEM FAILURE

PRODUCTION SERVERS

LOGS: ERROR 500

LOGS: ERROR 500

CRITICAL OVERLOAD!

KERNEL PANIC!

FIRE IN DATA CENTER!

BREAK GLASS  
IN CASE OF  
VIBE CHECK

CODE  
&  
VIBES

## 1.7 Interactive Activity: The "Vibe Coding" Challenge

### Classroom Poll & Discussion: AI in Practice

- **Poll:** When using AI coding assistants, how often do you inspect and understand every line before committing?
- **Discussion:** Suppose an AI assistant writes a 200-line asynchronous database handler that passes 2 basic tests. Is it safe to deploy? What verification steps must a professional engineer execute?





## 1. PUBLIC

Act consistently with the public interest.  
Prioritize safety, health, and welfare.



## 2. CLIENT & EMPLOYER

Act in the best interest of the client and employer.  
Maintain confidentiality.



## 3. PRODUCT

Ensure products meet high standards.  
Aim for quality, safety, and reliability.



## 4. JUDGMENT

Maintain integrity and independence.  
Exercise sound professional judgment.



## 5. MANAGEMENT

Use fair and effective management practices.  
Support ethical work environments.



## 6. PROFESSION

Advance the integrity and reputation of the profession.  
Report ethical violations.



## 7. COLLEAGUES

Be fair to and supportive of colleagues.  
Cooperate for shared success.



## 8. SELF

Participate in lifelong learning.  
Improve knowledge and skills continuously.

## 1.8 ACM/IEEE Code of Ethics: 8 Core Principles

1. **Public:** Prioritize public safety, health, and welfare above all.
2. **Client & Employer:** Act in their best interest, consistent with public interest.
3. **Product:** Ensure software meets high professional standards.
4. **Judgment:** Maintain integrity and independence in technical evaluation.
5. **Management:** Promote ethical management and realistic project estimates.
6. **Profession:** Advance the integrity and reputation of software engineering.
7. **Colleagues:** Be fair to, support, and mentor peers.
8. **Self:** Participate in lifelong learning and ethical practice.

## 1.8 High-Profile Ethical Breaches

- **Volkswagen "Dieselgate" (2015):**

- Engineers wrote engine software to detect laboratory test cycles and hide toxic  $NO_x$  emissions (up to 40x legal limit on the road).
- Resulted in billions in fines, criminal convictions, and severe environmental harm.

- **Cambridge Analytica (2018):**

- Improper harvesting of personal data for covert political manipulation.

- **Planned Obsolescence:**

- Software updates engineered to artificially degrade legacy device performance.

# DECEPTIVE UI/UX DARK PATTERNS



## 1.8 Deceptive "Dark Patterns" in UI/UX Design

- **1. Roach Motel (Subscription Labyrinth):**
  - Signing up takes 1 click; cancelling requires navigating hidden menus or making a phone call.
- **2. Confirmshaming:**
  - Emotionally manipulative text on decline buttons ("*No thanks, I hate saving money*").
- **3. Hidden Costs & Sneak into Basket:**
  - Pre-ticking add-on insurance or fees at the final checkout step.
- **4. Fabricated Urgency & Scarcity:**
  - Fake countdown timers ("*Only 2 minutes left!*") and fabricated demand alerts.

## Concept Check: Engineering Ethics (CCQ 6)

**Under the ACM/IEEE Code of Ethics, if an employer directs an engineer to implement an algorithm that falsifies safety compliance reports, what is the engineer's obligation?**

- **A.** Comply, because the employer pays the engineer's salary.
- **B.** Refuse and escalate, because the Public Interest takes precedence over Employer loyalty.
- **C.** Implement the code but omit documentation.
- **D.** Outsource the code to an external vendor.



## 1.8 Interactive Activity: Dark Pattern Detective

### Dark Pattern Detective Activity:

- **Identify:** Recall a deceptive dark pattern you encountered on a real-world app or e-commerce platform.
- **Analyze:** Which ACM/IEEE ethical principle (Public Interest, Product Quality, Professional Judgment) was violated?
- **Redesign:** How would you redesign that interaction to achieve legitimate business conversion while remaining transparent and user-respecting?



## 1.9 Frequently Asked Questions (FAQ)

- **Q1: Programming vs. Software Engineering?**

- *Answer:* Programming is writing code. Software engineering is programming integrated over time, managing team collaboration, constraints (budget/schedule), quality attributes, and long-term evolution.

- **Q2: Why do correct programs with 100% test coverage fail?**

- *Answer:* Software requires Data, Operational Procedures, and Documentation. Deficiencies in these non-code pillars or in Specification lead to failure.

- **Q3: Satisficing vs. Optimizing?**

- *Answer:* Real-world constraints (time, budget) require a "satisficing" solution (sufficient to meet constraints) rather than a technically "optimized" one.

- **Q4: The risk of "Vibe Coding" with AI?**

- *Answer:* Blindly accepting AI code without review leads to bugs. Prevent with code reviews, specification-first testing, and treating AI output as drafts.

## Recap: Fill-in-the-Blank Quiz

Test your mastery of Chapter 1 fundamentals:

1. According to the IEEE definition, software consists of programs, data, operational procedures, and [ \_\_\_\_\_ ].
2. Adding manpower to a late software project makes it later is known as [ \_\_\_\_\_ ] Law.
3. The [ \_\_\_\_\_ ] Quality Model defines 6 characteristics including Functionality, Reliability, Usability, Efficiency, Maintainability, and Portability.
4. The first principle of the ACM/IEEE Code of Ethics prioritizes the [ \_\_\_\_\_ ] interest.



# References & Further Reading

- Sommerville, Ian. *Software Engineering* (10th Edition). Pearson. [Official Website](#)
- Brooks, Frederick P. *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley.
- ISO/IEC 9126-1:2001. *Software engineering — Product quality — Part 1: Quality model*.
- ACM/IEEE Joint Task Force on Software Engineering Ethics. *Software Engineering Code of Ethics*. [IEEE CS](#)
- Brignull, Harry. *Deceptive Patterns: Exposing the Tricks Tech Companies Use to Control You*.